# Устранение несоответствий XML в TypeScript (для процессов локализации)

Родольфо М. Рая **Maxprograms** rmraya@maxprograms.com
17 июня 2026
© 2026 Maxprograms

*Описывает переход с Java на TypeScript для обработки XML в рабочих процессах локализации, а также библиотеки TypesXML, TypesBCP47 и TypesXLIFF, созданные для обеспечения валидации DTD и XML-схем для форматов TMX, XLIFF и SRX в Node.js.*

## Введение

Если вы когда-либо пытались обрабатывать XML в Node.js или TypeScript, вы, вероятно, сталкивались с одним ограничением: синтаксический анализ — дело несложное, а вот проверка на соответствие DTD или XML-схеме — нет.

Это становится настоящей проблемой при работе со стандартами локализации, такими как TMX, XLIFF или SRX, где структура и валидность не являются опциональными. Именно они определяют, пригодны ли ваши данные для использования.

Именно с этим пробелом я столкнулся и в итоге решил его устранить.

## Старая конфигурация: Java для XML

Стандарты локализации, такие как TMX, XLIFF, SRX и TBX, все основаны на XML. Когда я начал работать с ними более 20 лет назад, Java была наиболее практичным выбором. Её инструментарий для работы с XML не был идеальным, но достаточно надёжным для разбора и проверки на соответствие.

Долгое время это работало хорошо.

Проблема заключалась не в XML, а во всем, что его окружало.

Варианты пользовательского интерфейса для настольных приложений на Java не развивались так, как мне было нужно. Я много лет пользовался SWT, но поддержка этого фреймворка стала ухудшаться, а альтернативы, такие как Swing или JavaFX, меня не привлекали.

## Переход на TypeScript

Примерно в 2018 году я перешёл на создание пользовательских интерфейсов с помощью HTML, CSS и JavaScript. В конечном итоге TypeScript стал очевидным выбором, поскольку он обеспечивает более безопасную модель разработки благодаря статической проверке.

Это решило проблему с пользовательским интерфейсом.

Но это породило новую проблему:

- Пользовательский интерфейс на TypeScript
- Обработка XML по-прежнему в Java

Поддержка двух стеков для одного рабочего процесса была неидеальным решением.

## Недостающий элемент: валидация XML в TypeScript

В то время в JavaScript существовали XML-парсеры, но ни один из них не поддерживал:

- валидацию по DTD
- валидацию по XML-схеме

Для форматов локализации это обязательное требование. Без валидации невозможно гарантировать, что файл TMX или XLIFF структурно корректен.

Поэтому я начал разрабатывать **TypesXML**— собственный XML-парсер и валидатор для TypeScript.

## Минимальный пример

Вот как выглядит базовый разбор XML:

```
import { Catalog, DOMBuilder, SAXParser } from "typesxml";

const parser: SAXParser = new SAXParser();
const handler: DOMBuilder = new DOMBuilder();
const catalog = new Catalog('/pathTo/catalog.xml');

parser.setCatalog(catalog);
parser.setContentHandler(handler);
parser.setValidating(true);
parser.parseFile("example.xml");
```

```
const document = handler.getDocument();
console.log(document.toString());
```

Этот же парсер можно настроить для:

- проверку на соответствие DTD
- проверку на соответствие XML-схеме
- разрешать XML-каталоги

Это позволяет обрабатывать XML в Node.js с тем же уровнем контроля, который традиционно ассоциируется с библиотеками Java.

## Создание недостающего стека

Создание полного XML-стека заняло время:

- Сначала появились средства разбора в стиле DOM и SAX
- Затем последовала валидация DTD
- Проверка по XML-схеме стала последним шагом

Поддержка XML-схем значительно сложнее, но она необходима для таких форматов, как XLIFF.

## Вспомогательные элементы: языковые данные

Рабочие процессы локализации также в значительной степени зависят от кодов языков.

Стандарт определяется BCP47, но официальные данные распространяются в виде большого текстового файла. Чтобы упростить их использование в TypeScript, я создал **TypesBCP47**—вспомогательную библиотеку, которая анализирует эти данные и предоставляет их в структурированном виде.

## Подтверждение работоспособности концепции: переработка существующих инструментов

Чтобы проверить этот подход, я переписал существующие инструменты на TypeScript:

- **SRXEditor**
- **TMXValidator**

Эти инструменты:

- обрабатывают большие XML-файлы
- проверяют соответствие официальным грамматикам
- обеспечивают производительность, сопоставимую с исходными версиями на Java

Это подтвердило, что TypeScript способен справляться с реальными задачами локализации.

## Поддержка XLIFF

XLIFF использует XML-схему, что потребовало реализации полноценного валидатора схем.

После этого я создал **TypesXLIFF**— библиотеку для разбора, генерации и проверки файлов XLIFF 2.x.

Она поддерживает:

- проверку на соответствие XML-схеме
- проверку на соответствие спецификации XLIFF
- типизированную объектную модель
- преобразование в JSON

## Что это позволяет

Благодаря этим компонентам теперь можно:

- Проверять TMX, XLIFF и SRX непосредственно в Node.js
- Создавать конвейеры локализации полностью на TypeScript
- Избавиться от необходимости использовать отдельный бэкенд на Java
- Использовать один и тот же язык для интерфейса пользователя и обработки данных

## Что дальше

Ведётся работа над поддержкой TBX, а также обновлениями других инструментов с целью их полного перехода на TypeScript.

В старых компонентах по-прежнему присутствует некоторый код на Java, но новая разработка теперь сосредоточена на TypeScript.

## Заключительные мысли

Долгое время для обработки XML в процессах локализации фактически требовался Java.

Теперь это уже не так.

Если вы работаете с форматами на основе XML в Node.js, особенно в локализации или конвейерах структурированных данных, теперь вы можете полностью осуществлять разбор, валидацию и обработку в TypeScript.

## Об авторе



Родольфо М. Рая (Rodolfo M. Raya), технический директор компании Maxprograms SAS, участвует в разработке стандартов локализации, включая XLIFF, TMX, TBX, SRX и GlossML.

Он разрабатывает инструменты и библиотеки, реализующие эти спецификации, в том числе программное обеспечение и сервисы валидации, доступные на этом сайте.